

```
import pandas as pd
```

```
pip install -r requirements.txt
```

```
Requirement already satisfied: argon2-cffi>=21.1 in /usr/local/lib/python3.12/dist-packages (from jupyter-
Requirement already satisfied: jupyter-events>=0.9.0 in /usr/local/lib/python3.12/dist-packages (from jupy
Requirement already satisfied: jupyter-server-terminals>=0.4.4 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: nbconvert>=6.4.4 in /usr/local/lib/python3.12/dist-packages (from jupyter-s
Requirement already satisfied: nbformat>=5.3.0 in /usr/local/lib/python3.12/dist-packages (from jupyter-se
Requirement already satisfied: overrides>=5.0 in /usr/local/lib/python3.12/dist-packages (from jupyter-ser
Requirement already satisfied: prometheus-client>=0.9 in /usr/local/lib/python3.12/dist-packages (from jup
Requirement already satisfied: send2trash>=1.8.2 in /usr/local/lib/python3.12/dist-packages (from jupyter-
Requirement already satisfied: terminado>=0.8.3 in /usr/local/lib/python3.12/dist-packages (from jupyter-s
Requirement already satisfied: websocket-client>=1.7 in /usr/local/lib/python3.12/dist-packages (from jupy
Requirement already satisfied: platformdirs>=2.5 in /usr/local/lib/python3.12/dist-packages (from jupyter-
Requirement already satisfied: babel>=2.10 in /usr/local/lib/python3.12/dist-packages (from jupyterlab-ser
Requirement already satisfied: json5>=0.9.0 in /usr/local/lib/python3.12/dist-packages (from jupyterlab-se
Requirement already satisfied: jsonschema>=4.18.0 in /usr/local/lib/python3.12/dist-packages (from jupyter
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=
Requirement already satisfied: typing_extensions>=4.5 in /usr/local/lib/python3.12/dist-packages (from any
Requirement already satisfied: argon2-cffi-bindings in /usr/local/lib/python3.12/dist-packages (from argon
Requirement already satisfied: cffi>=1.12 in /usr/local/lib/python3.12/dist-packages (from cryptography>=3
Requirement already satisfied: jedi>=0.16 in /usr/local/lib/python3.12/dist-packages (from ipython>=7.23.1
Requirement already satisfied: decorator in /usr/local/lib/python3.12/dist-packages (from ipython>=7.23.1-
Requirement already satisfied: pickleshare in /usr/local/lib/python3.12/dist-packages (from ipython>=7.23.
Requirement already satisfied: prompt-toolkit!=3.0.0,!<3.0.1,<3.1.0,>=2.0.0 in /usr/local/lib/python3.12/d
Requirement already satisfied: pygments in /usr/local/lib/python3.12/dist-packages (from ipython>=7.23.1->
Requirement already satisfied: backcall in /usr/local/lib/python3.12/dist-packages (from ipython>=7.23.1->
Requirement already satisfied: pexpect>4.3 in /usr/local/lib/python3.12/dist-packages (from ipython>=7.23.
Requirement already satisfied: attrs>=22.2.0 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: referencing>=0.28.4 in /usr/local/lib/python3.12/dist-packages (from jsonsc
Requirement already satisfied: rpds-py>=0.7.1 in /usr/local/lib/python3.12/dist-packages (from jsonschema>
Requirement already satisfied: entrypoints in /usr/local/lib/python3.12/dist-packages (from jupyter-client
Requirement already satisfied: python-json-logger>=2.0.4 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: pyyaml>=5.3 in /usr/local/lib/python3.12/dist-packages (from jupyter-events
Requirement already satisfied: rfc3339-validator in /usr/local/lib/python3.12/dist-packages (from jupyter-
Requirement already satisfied: rfc3986-validator>=0.1.1 in /usr/local/lib/python3.12/dist-packages (from j
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.12/dist-packages (from nbconvert>=
Requirement already satisfied: bleach!=5.0.0 in /usr/local/lib/python3.12/dist-packages (from bleach[css]!
Requirement already satisfied: defusedxml in /usr/local/lib/python3.12/dist-packages (from nbconvert>=6.4.
Requirement already satisfied: jupyterlab-pygments in /usr/local/lib/python3.12/dist-packages (from nbconv
Requirement already satisfied: mistune<4,>=2.0.3 in /usr/local/lib/python3.12/dist-packages (from nbconver
Requirement already satisfied: nbclient>=0.5.0 in /usr/local/lib/python3.12/dist-packages (from nbconvert>
Requirement already satisfied: pandocfilters>=1.4.1 in /usr/local/lib/python3.12/dist-packages (from nbcon
Requirement already satisfied: fastjsonschema>=2.15 in /usr/local/lib/python3.12/dist-packages (from nbfor
Requirement already satisfied: pyprocess in /usr/local/lib/python3.12/dist-packages (from terminado>=0.8.
Requirement already satisfied: webencodings in /usr/local/lib/python3.12/dist-packages (from bleach!=5.0.0
Requirement already satisfied: tinycss2<1.5,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from bleac
Requirement already satisfied: pycparser in /usr/local/lib/python3.12/dist-packages (from cffi>=1.12->cryp
Requirement already satisfied: parso<0.9.0,>=0.8.4 in /usr/local/lib/python3.12/dist-packages (from jedi>=
Requirement already satisfied: fqdn in /usr/local/lib/python3.12/dist-packages (from jsonschema[format-non
Requirement already satisfied: isoduration in /usr/local/lib/python3.12/dist-packages (from jsonschema[for
Requirement already satisfied: jsonpointer>1.13 in /usr/local/lib/python3.12/dist-packages (from jsonschem
Requirement already satisfied: rfc3987-syntax>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from json
Requirement already satisfied: uri-template in /usr/local/lib/python3.12/dist-packages (from jsonschema[fo
Requirement already satisfied: webcolors>=24.6.0 in /usr/local/lib/python3.12/dist-packages (from jsonsche
Requirement already satisfied: wcwidth in /usr/local/lib/python3.12/dist-packages (from prompt-toolkit!=3.
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.12/dist-packages (from beautifulsou
Requirement already satisfied: lark>=1.2.2 in /usr/local/lib/python3.12/dist-packages (from rfc3987-syntax
Requirement already satisfied: arrow>=0.15.0 in /usr/local/lib/python3.12/dist-packages (from isoduration-
```

```
df=pd.read_csv('/content/Naukri Jobs Data.csv')
```

```
df.head()
```

	job_post	company	company_rating	company_review	exp_required	salary_offered	job_location	job_c
0	Information Technology Executive	Marriott	4.2	(2907 Reviews)	0-3 Yrs	Not disclosed	Bhopal	Tr
1	Associate Director-Information Technology(SAP)	Unacademy	3.4	(544 Reviews)	5-15 Yrs	Not disclosed	Noida, Greater Noida	Sup tin
2	Lead - Information Technology	EchoIndia	4.0	(20 Reviews)	11-13 Yrs	Not disclosed	Delhi / NCR	des RES
3	Information Technology - Project Manager	Tata International	4.1	(287 Reviews)	4-8 Yrs	Not disclosed	Mumbai (All Areas)	Sh ma
4	Information Technology Trainee	Stefanini	3.8	(75 Reviews)	2-7 Yrs	Not disclosed	Noida	The re exp

Next steps:

[Generate code with df](#)[New interactive sheet](#)

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32738 entries, 0 to 32737
Data columns (total 10 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   job_post                             32738 non-null  object
1   company                             32738 non-null  object
2   company_rating                       32738 non-null  float64
3   company_review                       32738 non-null  object
4   exp_required                         32738 non-null  object
5   salary_offered                       32738 non-null  object
6   job_location                         32738 non-null  object
7   job_description                      32738 non-null  object
8   required_skills                      32732 non-null  object
9   Posted_as_on_22_5_2022              32552 non-null  object
dtypes: float64(1), object(9)
memory usage: 2.5+ MB
```

```
df = df.dropna(subset=['required_skills', 'job_post'])
```

```
df["required_skills"] = df["required_skills"].apply(
    lambda x: [
        s.strip().lower().replace("_", " ").replace("-", " ")
        for s in str(x).replace("\\n", "\n").split("\n")
        if s.strip()
    ]
)

X = df["required_skills"].apply(lambda x: " , ".join(x) if isinstance(x, list) else x)
```

df["required_skills"][0]

```
['supervisor',
 'administration',
 'tactical planning',
 'executive information technology',
 'resource allocation',
 'server support',
 'scheduling',
 'management']
```

```
unique_titles = df["job_post"].nunique()
print(f"◆ Total unique job titles: {unique_titles}")
```

◆ Total unique job titles: 15200

```
import re

def normalize_job_title(title: str) -> str:
    """
    Cleans up job titles:
    - Removes seniority (Senior, Jr, Lead, etc.)
    - Removes numeric levels (I, II, III, 1, 2, 3)
    - Normalizes spacing
    - Keeps main title text (like 'Software Engineer', 'Data Scientist')
    """
    title = str(title).lower().strip()

    # Remove common seniority prefixes/suffixes
    title = re.sub(r'\b(senior|sr\.?|lead|principal|junior|jr\.?|entry[-\s]*level|intern|apprentice)\b', ' ', title)

    # Remove level indicators (I, II, III, 1, 2, 3, iv, v)
    title = re.sub(r'\b(i{1,3}|iv|v|vi{0,2}|[1-9])\b', ' ', title)

    # Remove extra hyphens, commas, parentheses
    title = re.sub(r'[-,()]/', ' ', title)

    # Collapse multiple spaces into one
    title = re.sub(r'\s+', ' ', title).strip()

    # Capitalize words for neatness
    title = title.title()

    return title
```

```
y = df['job_post'].apply(normalize_job_title)
```

```
unique_titles = y.nunique()
print(f"◆ Total unique job titles: {unique_titles}")
```

◆ Total unique job titles: 12898

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
X_train = X_train.apply(lambda x: ' '.join(x) if isinstance(x, list) else x)
X_test = X_test.apply(lambda x: ' '.join(x) if isinstance(x, list) else x)
```

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(
    stop_words='english',
    max_features=8000,
    ngram_range=(1, 3),          # include unigrams, bigrams, trigrams
    token_pattern=r'(?u)\b[a-zA-Z][a-zA-Z+\\-_. ]+[a-zA-Z]\b'
)

X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)
```

```
import numpy as np
import matplotlib.pyplot as plt

feature_names = vectorizer.get_feature_names_out()
```

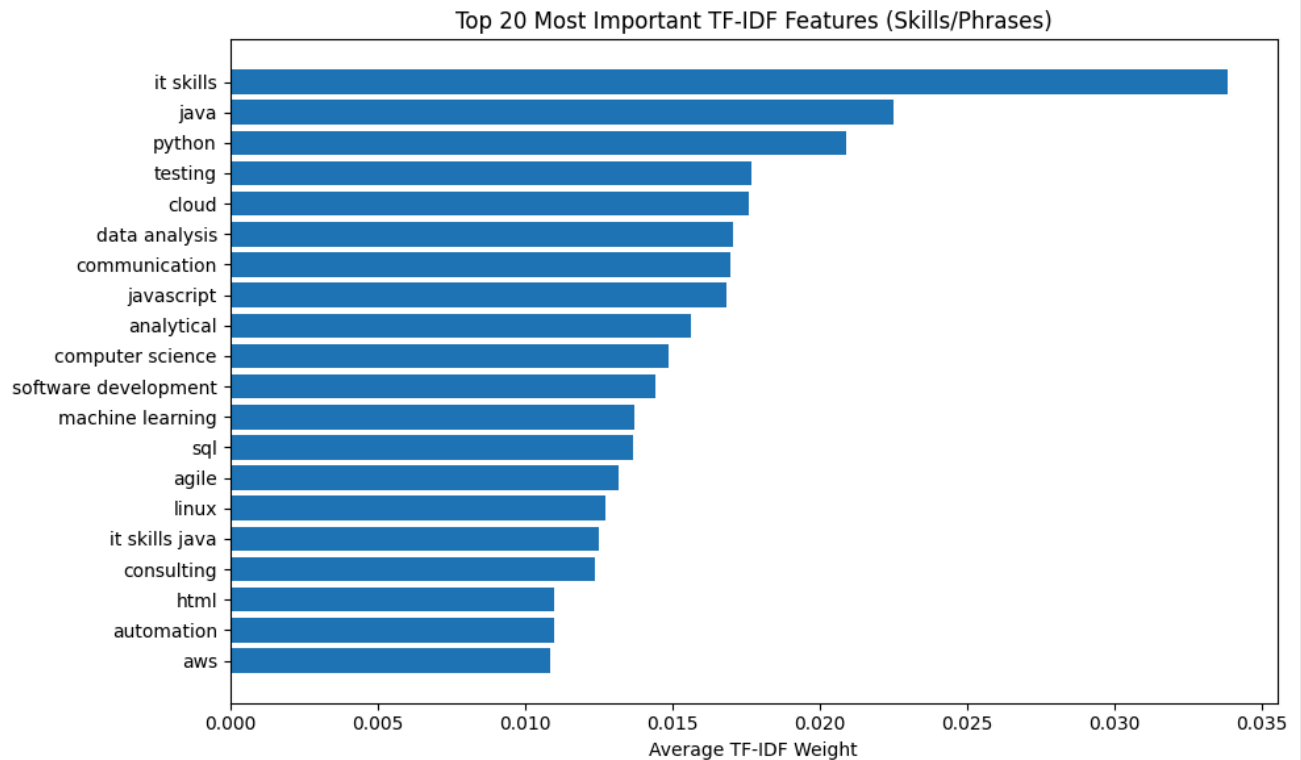
```

mean_tfidf = np.asarray(X_train_tfidf.mean(axis=0)).ravel()

top_n = 20
top_indices = mean_tfidf.argsort()[-top_n:][::-1]
top_features = feature_names[top_indices]
top_scores = mean_tfidf[top_indices]

plt.figure(figsize=(10, 6))
plt.barh(top_features[::-1], top_scores[::-1]) # reverse for visual ordering
plt.xlabel("Average TF-IDF Weight")
plt.title("Top 20 Most Important TF-IDF Features (Skills/Phrases)")
plt.tight_layout()
plt.show()

```



```
X_train_tfidf.shape
```

```
(26185, 8000)
```

```
len(set(y_train))
```

```
11061
```

```

def simplify_job_title(title):
    title = title.lower().strip()

    # Data & AI roles
    if any(w in title for w in ["data scientist", "research scientist", "quant"]):
        return "data_scientist"
    if any(w in title for w in ["machine learning", "ml engineer", "ai engineer", "deep learning"]):
        return "ml_engineer"
    if any(w in title for w in ["data engineer", "etl", "pipeline", "big data"]):
        return "data_engineer"
    if "analyst" in title or "business intelligence" in title:
        return "data_analyst"

```

```

# Software engineering roles
if any(w in title for w in ["software engineer", "sde", "backend", "full stack", "full-stack"]):
    return "software_engineer"
if any(w in title for w in ["frontend", "front-end", "ui developer", "react developer"]):
    return "frontend_engineer"
if any(w in title for w in ["devops", "site reliability", "sre", "cloud engineer"]):
    return "devops_engineer"
if any(w in title for w in ["mobile developer", "android", "ios", "flutter", "react native"]):
    return "mobile_engineer"

# Product / Management roles
if any(w in title for w in ["product manager", "pm", "program manager"]):
    return "product_manager"
if any(w in title for w in ["project manager", "scrum master"]):
    return "project_manager"

# Cybersecurity roles
if any(w in title for w in ["security engineer", "cybersecurity", "infosec"]):
    return "security_engineer"

# Cloud roles
if any(w in title for w in ["aws", "azure", "gcp", "cloud architect"]):
    return "cloud_architect"

# QA / Testing roles
if any(w in title for w in ["qa", "test engineer", "automation engineer"]):
    return "qa_engineer"

return "other"
y_train = y_train.apply(simplify_job_title)
y_test = y_test.apply(simplify_job_title)

```

```

from sklearn.linear_model import SGDClassifier

model = SGDClassifier(
    loss="log_loss",      # logistic regression style
    max_iter=1000,
    tol=1e-3,
    n_jobs=-1,
    learning_rate="optimal",
    alpha=1e-4           # regularization strength
)
model.fit(X_train_tfidf, y_train)

```

▼ SGDClassifier ⓘ ?

```
SGDClassifier(loss='log_loss', n_jobs=-1)
```

```

from sklearn.metrics import accuracy_score, f1_score

print(
    f"Accuracy: {accuracy_score(y_test, model.predict(X_test_tfidf)):.4f} | "
    f"Weighted F1: {f1_score(y_test, model.predict(X_test_tfidf), average='weighted'):.4f} | "
    f"Top-5 Acc: {np.mean([y in model.classes_[np.argsort(p)[-5:]] for y, p in zip(y_test, model.predict_p
)

```

Accuracy: 0.6003 | Weighted F1: 0.5607 | Top-5 Acc: 0.9716

```

def get_top_skills_per_job(model, vectorizer, top_n=20):
    """Extract most important skill tokens for each job role."""
    feature_names = np.array(vectorizer.get_feature_names_out())
    job_skill_map = {}
    for i, job in enumerate(model.classes_):
        coefs = model.coef_[i]
        top_indices = np.argsort(coefs)[::-1][:top_n]
        job_skill_map[job] = list(feature_names[top_indices])

```

```
return job_skill_map
```

```
job_top_skills = get_top_skills_per_job(model, vectorizer, top_n=20)
```

```
def suggest_jobs(skills_text, top_n=5):
    """Predict top job matches with probabilities."""
    X_input = vectorizer.transform([skills_text])
    probs = model.predict_proba(X_input)[0]
    sorted_idx = np.argsort(probs)[-1:][:top_n]
    results = [(model.classes_[i], round(probs[i] * 100, 2)) for i in sorted_idx]
    return results
```

```
import re

def clean_skill_text(text):
    text = re.sub(r'([a-z])([A-Z])', r'\1 \2', text)
    text = text.lower()
    text = text.replace("_", " ").replace("-", " ").replace("\n", " ")
    text = " ".join(text.split())
    return text

def get_job_suggestions(skills_input, top_n=5):
    if isinstance(skills_input, (list, tuple, set)):
        cleaned_skills = [clean_skill_text(s) for s in skills_input]
        skills_text = " , ".join(cleaned_skills)
    else:
        skills_text = clean_skill_text(str(skills_input))

    user_skills = set([s.strip() for s in skills_text.replace(",", " , ").split(" , ") if s.strip()])

    results = suggest_jobs(skills_text, top_n)

    print(f"\n📁 **Top {top_n} Job Matches:**\n")
    for job, score in results:
        print(f"💎 {job} ({score}%)")

        ideal_skills = set([" ".join(skill.split()) for skill in job_top_skills.get(job, [])])

        learned = set()
        missing = set()

        for skill in ideal_skills:
            if any(skill in u or u in skill for u in user_skills):
                learned.add(skill)
            else:
                missing.add(skill)
        if ideal_skills:
            match_pct = round((len(learned) / len(ideal_skills)) * 100, 1)
            print(f"✅ Skill Match: {match_pct}% ({len(learned)}/{len(ideal_skills)} skills)")
            if learned:
                print(f"🧠 You already know: {' , '.join(sorted(learned))}")
            if missing:
                print(f"🚀 Learn next: {' , '.join(sorted(missing))}")
            else:
                print(f"🎉 You already have all the top skills for this role!")
        else:
            print(f"(No top skill data found for this role.)")

    print()
    return results
```

```
get_job_suggestions([
    "Python", "R", "Java", "SQL", "Typescript", "HTML5", "CSS", "Tableau", "Power BI", "Git",
    "React", "Express.js", "Node.js", "Smart Contracts", "PostgreSQL",
    "Object-Oriented Programming", "Data Analysis", "Data Visualization",
    "Machine Learning", "Neural Networks", "Deep Learning", "Cloud Computing",
    "Algorithms", "System architecture", "Database Management Systems",
```

```

    "Data Structures", "Software Engineering", "Information Retrieval Systems",
    "Web Development", "Quantum Computing", "Feature Scaling", "Collaboration",
    "Problem Solving", "Analytical Thinking", "Communication", "Time Management",
    "Critical Thinking", "Detail Orientation", "Adaptability"
], top_n=5)

```

 ****Top 5 Job Matches:****

```

◆ software_engineer (45.56%)
  ✓ Skill Match: 75.0% (15/20 skills)
  🧠 You already know: angular, azure, computer science, data structures, embedded software, html, java, j
  🚀 Learn next: agile, backend, coding, debugging, unit testing

◆ other (16.51%)
  ✓ Skill Match: 80.0% (16/20 skills)
  🧠 You already know: application support, aws azure, business process, data analysis data management, de
  🚀 Learn next: consulting, lan, sap, wan

◆ data_scientist (9.48%)
  ✓ Skill Match: 55.0% (11/20 skills)
  🧠 You already know: analytical machine learning, artificial intelligence, computer science, deep learni
  🚀 Learn next: data mining, data science, data science analytical, data scientist, nlp, sas, statistical

◆ data_analyst (6.54%)
  ✓ Skill Match: 40.0% (8/20 skills)
  🧠 You already know: analytical, dashboards, data analysis, financial services, power bi, senior analyst
  🚀 Learn next: analyst, analytics, business analysis, business analyst, business intelligence, data anal

◆ data_engineer (6.25%)
  ✓ Skill Match: 55.0% (11/20 skills)
  🧠 You already know: data engineer, data engineering, data integration, data migration, data warehousing
  🚀 Learn next: agile, big data, business intelligence, data modeling, datastage, etl, hadoop, san, scala

[(np.str_('software_engineer'), np.float64(45.56)),
 (np.str_('other'), np.float64(16.51)),
 (np.str_('data_scientist'), np.float64(9.48)),
 (np.str_('data_analyst'), np.float64(6.54)),
 (np.str_('data_engineer'), np.float64(6.25))]

```

EXPLANATION:

The “data_engineer” label (and similar ones like “software_engineer”, “ml_engineer”, etc.)

is not a single, narrow job title — it represents a *cluster* of closely related sub-roles

that share many overlapping skills. For example:

data_engineer → { ETL Developer, Big Data Engineer, Data Pipeline Engineer, Hadoop Engineer }

When the model predicts a “Job Match %” for this label:

→ It reflects how close your *overall skill pattern* (as learned by the TF-IDF + classifier)

is to the *average skill distribution* across all roles inside that cluster.

The "Skill Match %" on the other hand:

→ Simply measures how many of the top skills for that cluster (e.g., Spark, ETL, Hadoop, Python)

appear directly in your own skill list.

So if you ever see a lower Job Match % but a higher Skill Match %, that means your skills overlap strongly with the cluster, but your *overall skill pattern* (frequency and combination of skills) more closely resembles a different role.

In short:

- Job Match % → Statistical similarity to the cluster's average skill pattern.
- Skill Match % → Literal overlap between your skills and the cluster's key skills.